

Web программирование

Курс для бакалавриата

Александр Менщиков,
к.т.н., доцент ИТМО

Реактивный фронтенд

Александр Менщиков,
к.т.н., доцент ИТМО

Алексей Вохмин

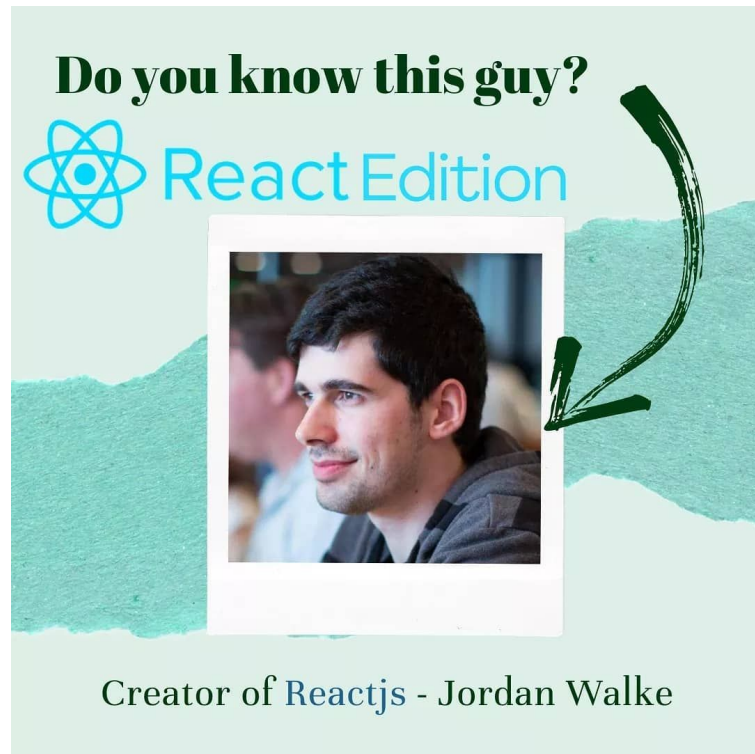
Содержание лекции

1. История React
2. Зачем нужен React?
3. Основы React и его компоненты
4. JSX
5. Жизненный цикл компонентов
6. Состояние и свойства (State и Props)
7. Хуки (Hooks)
8. Бандлеры (Bundlers)

История React

Создание React

- Создан в 2011 Джорданом Уолке для нужд Facebook
- React улучшает производительность и упрощает поддержку интерфейсов
- Открыт для широкой публики в 2013 году на JSConf US



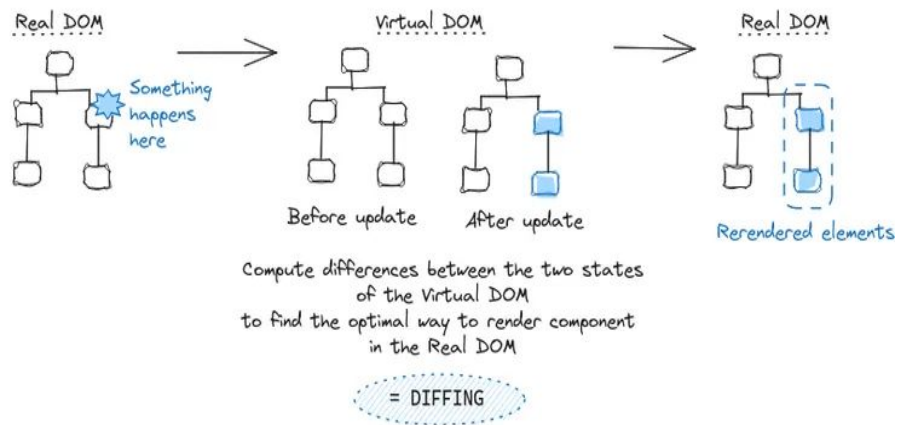
Эволюция библиотеки и важные версии

- 2013 - 0.3.0: первая публичная версия React
- 2015 - 0.13.0: поддержка ES6 классов
- 2017 - 16.0.0: представление "границ ошибок" (error boundaries) для создания более устойчивых приложений
- 2018 - 16.8.0: добавлены React Hooks, изменившие подход к управлению состоянием
- 2020 - 17.0.0: подготовка к постепенному обновлению версий, JSX Transform
- 2022 - 18.0.0: добавление Concurrent React, автоматическое группирование обновлений
- 2024 - 19.0.0-rc: новые хуки, серверные компоненты

Зачем нужен React?

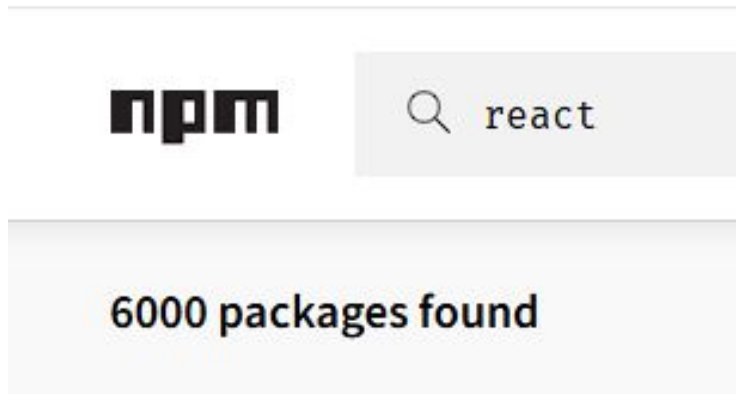
Проблемы, которые решает React

- **Проблемы производительности** — Virtual DOM для минимизации обновлений.
- **Сложность поддержки кода** — Модульный компонентный подход.
- **Повторяемость элементов** — Переиспользуемые компоненты.



Почему React?

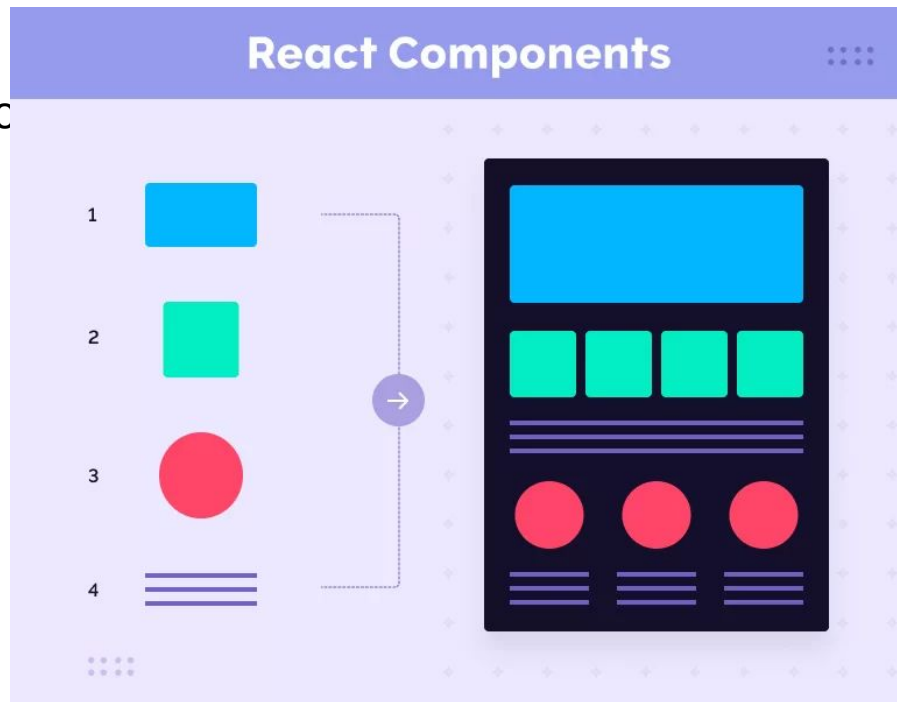
- Интуитивный API и документация
- Подходит для малых и крупных приложений
- Библиотеки, готовые решения, поддержка



Основы React и его компоненты

Компоненты в React

- Компоненты - это строительные блоки интерфейсов, которые можно использовать повторно
- Компоненты бывают функциональные и классовые
- Преимущества: модульность, переиспользуемость, легкость поддержки



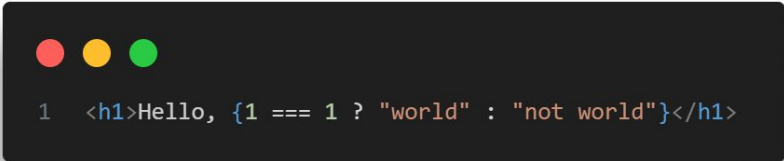
Функциональные и классовые компоненты



```
1  function FunctionalComponent() {  
2    return <h1>Hello, world</h1>;  
3  }  
4  
5  class ClassComponent extends React.Component {  
6    render() {  
7      return <h1>Hello, world</h1>;  
8    }  
9  }  
10
```

JSX

Что такое JSX?



```
1 <h1>Hello, {1 === 1 ? "world" : "not world"}</h1>
```

- Является синтаксическим расширением JavaScript, напоминающим HTML
- Упрощает работу с React-компонентами
- Определяет структуру интерфейса прямо в коде JavaScript

Как работает JSX?

```
1 function InnerComponent() {  
2   return <>World</>  
3 }  
4  
5 function Component() {  
6   return <h1>Hello <InnerComponent /> {1 === 1}</h1>;  
7 }  
8
```



- JS

```
function InnerComponent() {  
  const $ = _c(1);  
  let t0;  
  if ($[0] === Symbol.for("react.memo_cache_sentinel")) {  
    t0 = <>World</>;  
    $[0] = t0;  
  } else {  
    t0 = $[0];  
  }  
  return t0;  
}  
  
function Component() {  
  const $ = _c(1);  
  let t0;  
  if ($[0] === Symbol.for("react.memo_cache_sentinel")) {  
    t0 = (  
      <h1>  
        | Hello <InnerComponent /> {true}  
      </h1>  
    );  
    $[0] = t0;  
  } else {  
    t0 = $[0];  
  }  
  return t0;  
}
```

Как работает JSX?

```
9318 function InnerComponent() {
9319   return /* #__PURE__ */ jsxRuntimeExports.jsx(jsxRuntimeExports.Fragment, {
9320     children: "World",
9321   });
9322 }
9323 function App() {
9324   return /* #__PURE__ */ jsxRuntimeExports.jsx("h1", {
9325     children: [
9326       "Hello ",
9327       /* #__PURE__ */ jsxRuntimeExports.jsx(InnerComponent, {}),
9328       " ",
9329       true,
9330     ],
9331   });
9332 }
9333 createRoot(document.getElementById("root")).render(
9334   /* #__PURE__ */ jsxRuntimeExports.jsx(reactExports.StrictMode, {
9335     children: /* #__PURE__ */ jsxRuntimeExports.jsx(App, {}),
9336   })
9337 );
9338
```

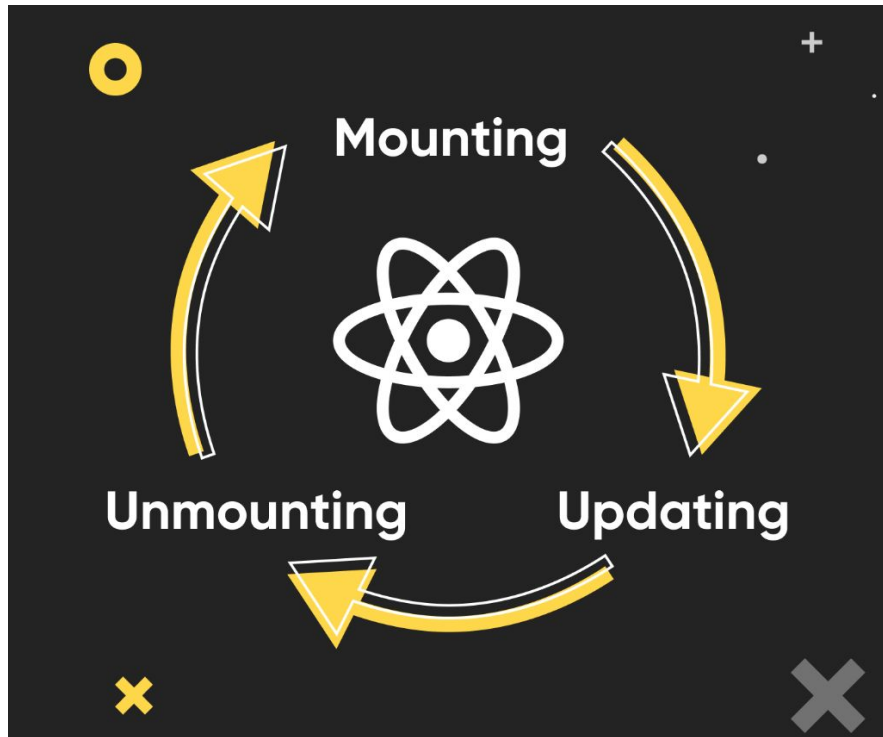

Жизненный цикл компонентов

Жизненный цикл компонентов в React

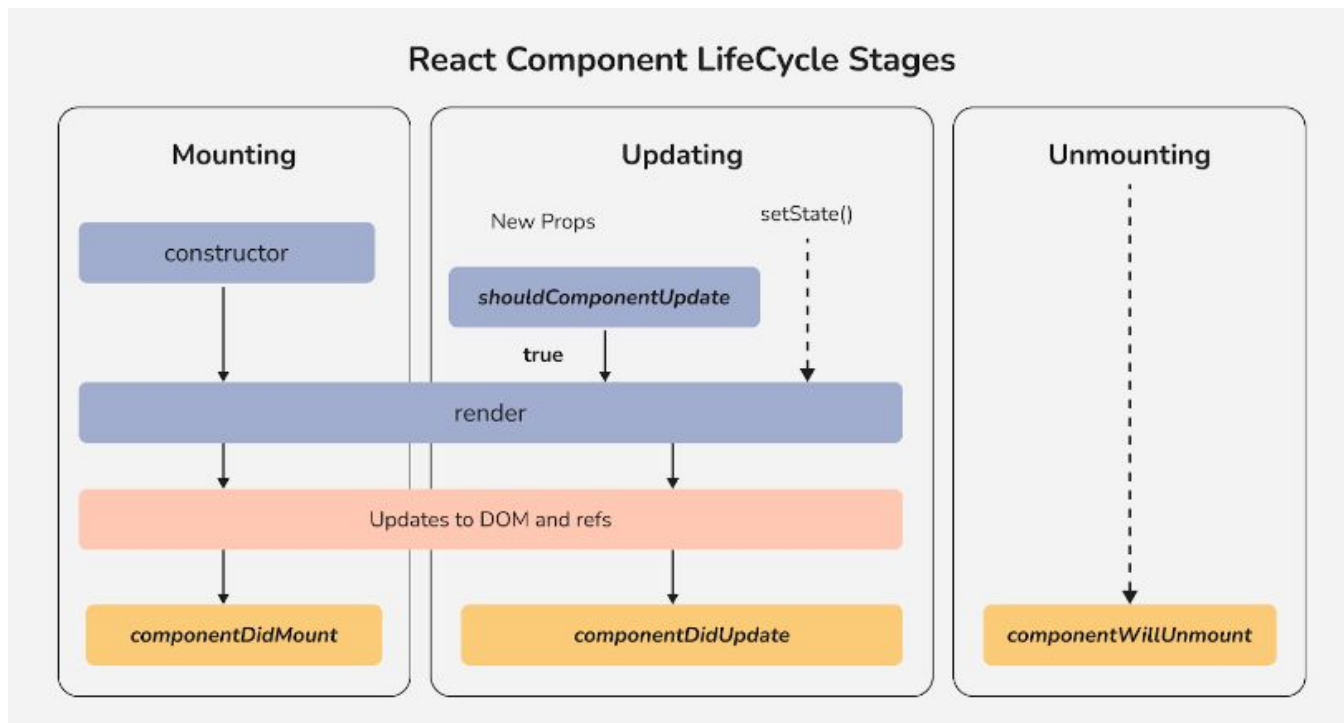
Жизненный цикл — это этапы, через которые проходит компонент от создания до удаления

Основные фазы:

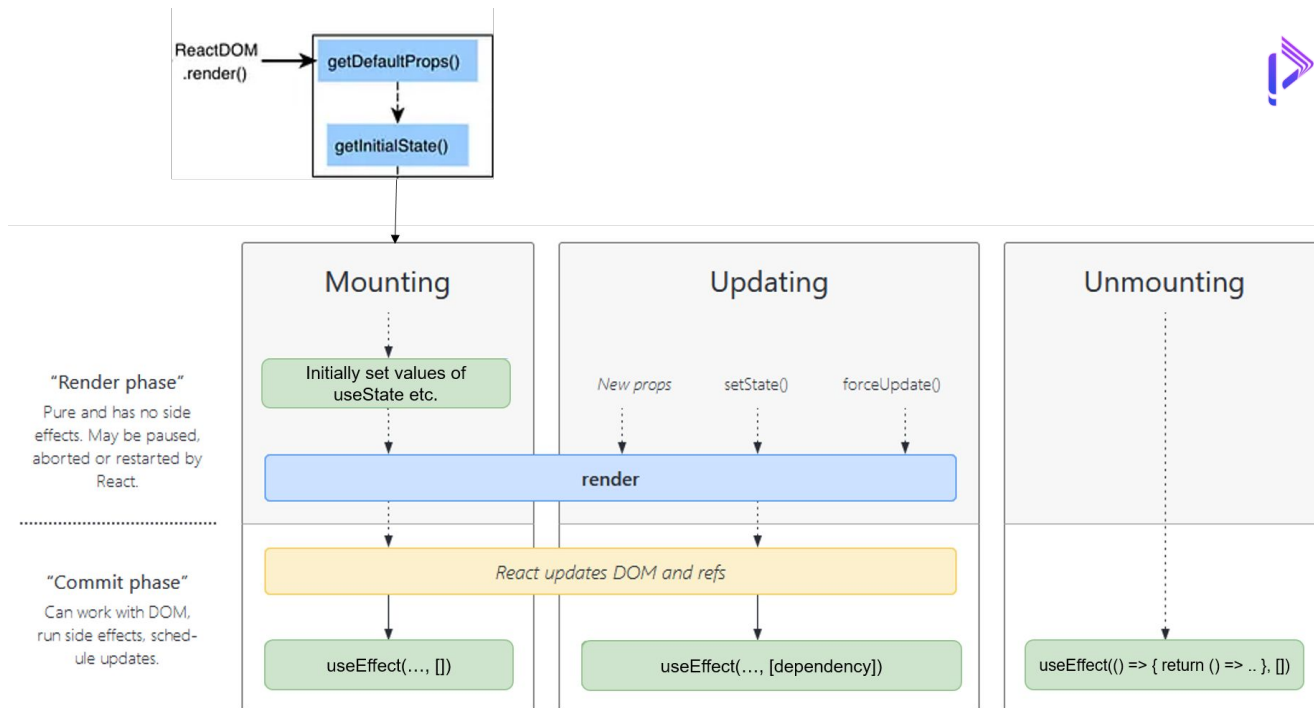
- **Монтирование:** создание и добавление компонента на страницу
- **Обновление:** перерисовка компонента при изменении props или state
- **Размонтирование:** удаление компонента со страницы



Основные методы жизненного цикла компонентов



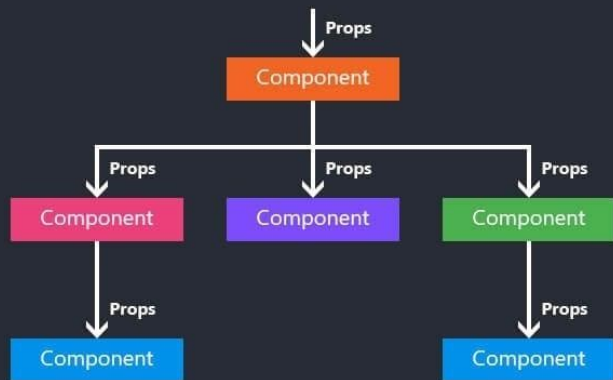
Жизненный цикл функциональных компонентов



Состояние и свойства (State и Props)

Что такое Props?

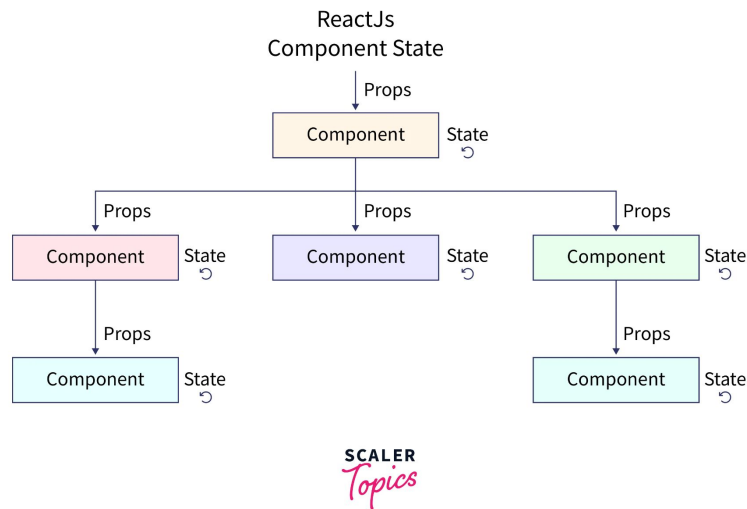
Understanding ReactJS Props



- Props - это данные, передаваемые компоненту извне
- Компонент не может изменить полученные props
- Используются для передачи данных и настроек

Что такое State?

- State - это данные, которые контролируются внутри компонента
- Компонент может обновлять своё состояние
- Обновление state вызывает перерисовку компонента



Взаимодействие Props и State

```
1 import { useState } from "react";
2
3 function InnerComponent({
4   value,
5   setValue,
6 }): {
7   value: string;
8   setValue: (value: string) => void;
9 } {
10  return (
11    <input value={value} onChange={(event) => setValue(event.target.value)} />
12  );
13 }
14
15 function App() {
16   const [value, setValue] = useState("Hello World");
17
18   return <InnerComponent value={value} setValue={setValue} />;
19 }
```

```
1 import { useState } from "react";
2
3 function Button({ text }: { text: string }) {
4   const [disabled, setDisabled] = useState(false);
5
6   return (
7     <button disabled={disabled} onClick={() => setDisabled(!disabled)}>
8       {text}
9     </button>
10   );
11 }
12
13 function App() {
14   return <Button text="Hello World" />;
15 }
```


Хуки (Hooks)

Что такое хуки в React?

Хуки - это функции, которые позволяют использовать состояние и другие возможности React в функциональных компонентах

Основные хуки:

- `useState` - управление состоянием
- `useEffect` - побочные эффекты
- `useContext` - доступ к данным контекста

useState и useEffect

useState:

- Инициализация состояния: `const [state, setState] = useState(initialValue)`
- Обновление состояния с помощью функции

useEffect:

- Запуск кода при монтировании, обновлении или размонтировании компонента
- Управление зависимостями с помощью массива зависимостей

useContext: доступ к контексту

useContext — хук для получения значений из контекста в функциональных компонентах

Упрощает доступ к данным, избегая вложенной структуры потребителей

`const value = useContext(MyContext);` извлекает значение из контекста `MyContext`

Эволюция библиотеки и важные версии

<https://react.dev/reference/react/hooks>



Hooks



useActionState 

useCallback

useContext

useDebugValue

useDeferredValue

useEffect

useId

useImperativeHandle

useInsertionEffect

useLayoutEffect

useMemo

useOptimistic 

useReducer

useRef

useState

useSyncExternalStore

useTransition

Бандлеры (Bundlers)

Что такое JavaScript-бандлеры?

Бандлеры — это инструменты для объединения множества файлов JavaScript в один или несколько пакетов, что помогает ускорить загрузку приложений

Они оптимизируют код, уменьшают количество запросов к серверу и поддерживают новые стандарты JavaScript, что облегчает работу разработчиков

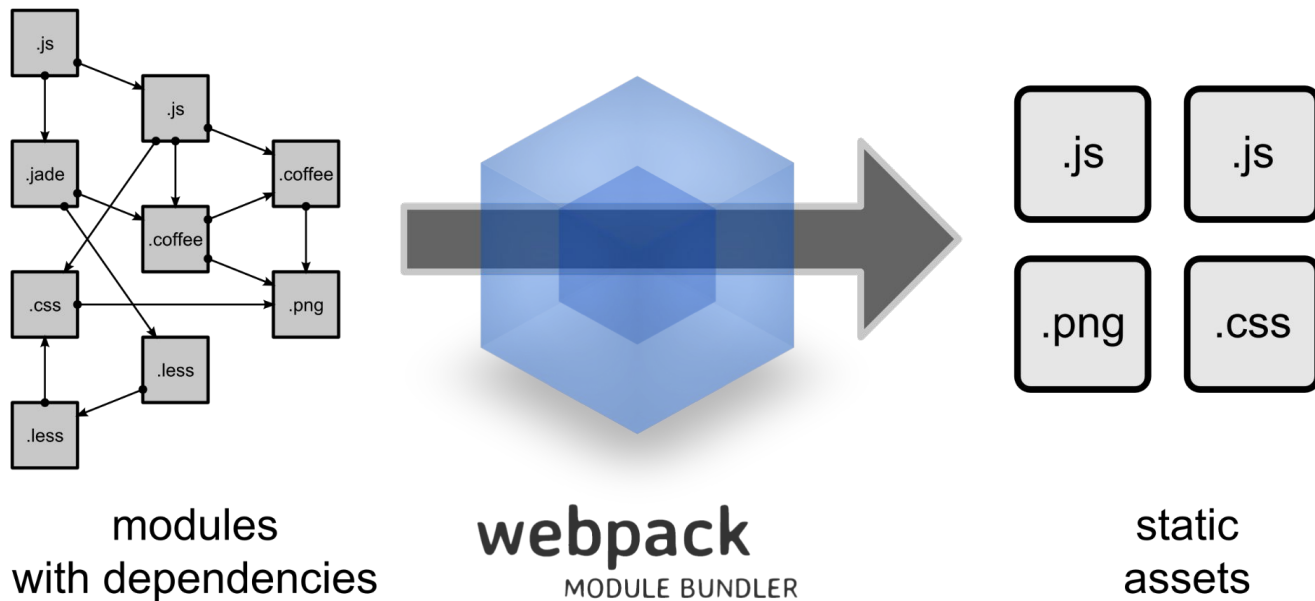


```
<head>
  <script src="https://unpkg.com/@popperjs/core@2/dist/umd/popper.min.js"></script>
  <script type='text/javascript' src='tinycolor.js'></script>
  <script type='text/javascript' src='/nav.js'></script>
  <script type='text/javascript' src='/crud.js'></script>
  <script type='text/javascript' src='/main.js'></script>
</head>
```



```
<head>
  <script type='text/javascript' src='/bundle.js'></script>
</head>
```

Как работают бандлеры



Webpack

Да:

- Поддержка любых типов ресурсов
- Богатая экосистема плагинов и лоадеров
- Широкие возможности оптимизации
- Отличная документация

Но:

- Сложная настройка
- Может быть медленным

Browserify

Да:

- Объединяет Node.js модули для использования в браузере
- Поддерживает require() для npm-пакетов
- Простая интеграция с npm
- Быстрая сборка и отличная документация

Но:

- Нет поддержки нескольких типов ресурсов
- Ограниченные возможности управления зависимостями

Parcel

Да:

- Автоматическая сборка JS, CSS, HTML и других ресурсов
- Поддержка нескольких точек входа
- Высокая скорость сборки за счет кэширования и оптимизаций
- Простота использования: `npm i parcel` и `parcel index.html`

Но:

- Ограниченные возможности кастомизации для сложных проектов

FuseBox

Да:

- Минимальная конфигурация, лёгкий старт
- Высокая скорость сборки, HMR, кэширование
- Богатый API

Но:

- Слабая поддержка ресурсов, отличных от JS/TS
- Небольшое сообщество

Rollup

Да:

- Удаляет неиспользуемый код (tree-shaking)
- Создает компактные бандлы
- Поддержка ES6 модулей
- Оптимизация ресурсов

Но:

- Экосистема плагинов ещё развивается

esbuild

Да:

- Скорость: В разы быстрее других бандлеров
- Простота: Минималистичная конфигурация
- Поддержка: JS, CSS, TypeScript, JSX/TSX
- API: Для интеграции в другие инструменты

Но:

- Ограниченная функциональность по сравнению с Webpack

Vite

Да:

- Мгновенная загрузка с помощью ES-модулей
- Горячая замена модулей (HMR)
- Поддержка популярных фреймворков (React, Vue, etc.)
- Богатая экосистема плагинов

Но:

- Проблемы совместимости со старыми браузерами

Coding samples

https://github.com/cs-itmo/webdev_2024



Вопросы?

Эволюция библиотеки и важные версии

Содержание 1

Полезные ссылки

- <https://learn.javascript.ru/closure>